U24AI006

ASSIGNMENT-14

1. Write a program to solve the questions using Forward Chaining.

a. Knowledge Base:
$P \rightarrow Q$
$L \wedge M \rightarrow P$
$A \wedge B \rightarrow L$
A
B
M
Conclusion:
Q

b. Knowledge Base:
$A \rightarrow B$
$B \rightarrow C$
$C \rightarrow D$
A
E
$D \wedge E \rightarrow F$
Conclusion: F

2. Write a program to solve the questions using Backward Chaining.

a. Knowledge Base:
$P \rightarrow Q$
$R \rightarrow Q$
$A \rightarrow P$
$B \rightarrow R$
A
B
Conclusion:
Q

b. Knowledge Base:
$A \rightarrow B$
$B \wedge C \rightarrow D$
$E \rightarrow C$
A
E
Conclusion:
D

3. Write a program to solve the questions using the method Resolution and proof of refutation/contradiction.

a. Knowledge Base:
$P \vee Q$
$P \rightarrow R$
$Q \rightarrow S$
$R \rightarrow S$
Conclusion
S

b. Knowledge Base:
$P \rightarrow Q$
$Q \rightarrow R$
$S \rightarrow \neg R$
P
Conclusion:
S

CODE(1):

```
def forward_chaining(facts, rules, goal):
    known = facts[:]
    changed = True

    while changed:
        changed = False

        for conditions, result in rules:
            all_present = True

            for condition in conditions:
                if condition not in known:
                    all_present = False
                    break

            if all_present and result not in known:
                known.append(result)
                print(result)
                changed = True

                if result == goal:
                    return True, known

    return goal in known, known

def solve(name, facts, rules, goal):
    print(name)
    print("Facts:", ", ".join(facts))
```

```
    found, known = forward_chaining(facts, rules, goal)

    if found:
        print(goal, "is proved")
    else:
        print(goal, "is not proved")

    print("Final facts:", ", ".join(known))
    print()

rules_a = [
    (["P"], "Q"),
    (["L", "M"], "P"),
    (["A", "B"], "L")
]

facts_a = ["A", "B", "M"]

rules_b = [
    (["A"], "B"),
    (["B"], "C"),
    (["C"], "D"),
    (["D", "E"], "F")
]

facts_b = ["A", "E"]

solve("a", facts_a, rules_a, "Q")
solve("b", facts_b, rules_b, "F")
```

OUTPUT:

```
a
To prove Q
To prove P
A is a fact
P is proved
Q is proved
Conclusion: Q is true

b
To prove D
To prove B
A is a fact
B is proved
To prove C
E is a fact
C is proved
D is proved
Conclusion: D is true
```

CODE(2):

```
def backward_chaining(goal, facts, rules, checked):
    if goal in facts:
        print(goal, "is a fact")
        return True

    if goal in checked:
        return False

    checked.append(goal)

    for conditions, result in rules:
        if result == goal:
            print("To prove", goal)
            all_true = True

            for condition in conditions:
                if not backward_chaining(condition, facts, rules, checked):
                    all_true = False
                    break

            if all_true:
                print(goal, "is proved")
                return True

    return False

def solve(name, facts, rules, goal):
    print(name)

    if backward_chaining(goal, facts, rules, []):
        print("Conclusion:", goal, "is true")
    else:
        print("Conclusion:", goal, "is false")

    print()

rules_a = [
    (["P"], "Q"),
    (["R"], "Q"),
    (["A"], "P"),
    (["B"], "R")
]

facts_a = ["A", "B"]
```

```
rules_b = [
    (["A"], "B"),
    (["B", "C"], "D"),
    (["E"], "C")
]

facts_b = ["A", "E"]

solve("a", facts_a, rules_a, "Q")
solve("b", facts_b, rules_b, "D")
```

OUTPUT:

```
a
To prove Q
To prove P
A is a fact
P is proved
Q is proved
Conclusion: Q is true

b
To prove D
To prove B
A is a fact
B is proved
To prove C
E is a fact
C is proved
D is proved
Conclusion: D is true
```

CODE(3):

```
def neg(value):
    if value[0] == "~":
        return value[1:]
    return "~" + value

def clause_text(clause):
    if len(clause) == 0:
        return "{}"
    return " v ".join(sorted(clause, key=lambda value: value.replace("~", "")))

def has_opposite(clause):
    for value in clause:
        if neg(value) in clause:
            return True
    return False

def resolve(first, second):
    answers = []

    for value in first:
        opposite = neg(value)

        if opposite in second:
            new_clause = set(first)
            new_clause.remove(value)

            for item in second:
                if item != opposite:
                    new_clause.add(item)

            if not has_opposite(new_clause):
                answers.append(frozenset(new_clause))

    return answers

def resolution(name, clauses, goal):
    print(name)

    all_clauses = []
    for clause in clauses:
        all_clauses.append(frozenset(clause))

    all_clauses.append(frozenset([neg(goal)]))
```

```
    print("Clauses")
    for clause in all_clauses:
        print(clause_text(clause))

    print("Resolving")

    while True:
        added = []

        for i in range(len(all_clauses)):
            for j in range(i + 1, len(all_clauses)):
                new_clauses = resolve(all_clauses[i], all_clauses[j])

                for new_clause in new_clauses:
                    if len(new_clause) == 0:
                        print("Empty clause")
                        print("Conclusion:", goal, "is proved")
                        print()
                        return

                    if new_clause not in all_clauses and new_clause not in added:
                        added.append(new_clause)
                        print(clause_text(new_clause))

        if len(added) == 0:
            print("No contradiction")
            print("Conclusion:", goal, "is not proved")
            print()
            return

        for clause in added:
            all_clauses.append(clause)

clauses_a = [
    ["P", "Q"],
    ["~P", "R"],
    ["~Q", "S"],
    ["~R", "S"]
]

clauses_b = [
    ["~P", "Q"],
    ["~Q", "R"],
    ["~S", "~R"],
    ["P"]
```

```
]

resolution("a", clauses_a, "S")
resolution("b", clauses_b, "S")
```

OUTPUT:

```
a
Clauses
P v Q
~P v R
~Q v S
~R v S
~S
Resolving
Q v R
P v S
~P v S
~Q
~R
Q v S
P
R v S
~P
R
Q
S
Empty clause
Conclusion: S is proved

b
Clauses
~P v Q
~Q v R
~R v ~S
P
~S
Resolving
~P v R
Q
~Q v ~S
~P v ~S
R
No contradiction
Conclusion: S is not proved
```